
Debugging Lab Material

Course: *Introduction to Vulnerability Research Lab*

Author name: *Dr. Bernard Lampe*

Due date: *November 14th, 2019*

Setup

1. Reuse two verified targets from earlier labs:

- `source\_auditing/heap.c` (string store; UAF on double DELETE)
- `fuzzing/targets/lab\_target.c + harness.c + main.c` (CKS parser; stack overflow at `lab\_target.c:46`)

Build both plain (no sanitizer) — this lab observes the *unsanitized* program:

```
1 clang -g -O0 ../../source_auditing/heap.c -o heap_plain
2 clang -g -O0 lab_target.c harness.c main.c -o lab_plain
```

2. Debugger on this reference host is `lldb`; `gdb` commands are given in comments where they differ.
3. Also prepare the fuzzing-lab seeds (`seed\_over`, the CREATE/DELETE driver from the source auditing lab Exercise A).

Exercise 1 — Watchpoint Data Tracing

1. Run `lab\_plain seed\_over` under `lldb`:

```
1 lldb -- ./lab_plain seed_over
2 (lldb) b lab_parse          # gdb: break lab_parse
3 (lldb) run
```

2. Verified entry state on the reference build: at the first stop, `x1` holds size (`0x135 = 309` for `seed\_over`) and `x0` points into `main`'s file-read buffer.
3. Step to line 46 (`b lab\_target.c:46`, then `continue`), evaluate `(uint64\_t)&buf[64]` (verified: `= 0x16fdfe678` on this build), and set a hardware write watchpoint on that byte:

```
1 # lldb
2 watchpoint set expression -w write -s 1 \
3   -- *(unsigned char *)((uintptr_t)&buf[64])
4 # gdb
5 watch *(char*)<addr>
```

4. Observed result (verified): the plain SDK build emits `\\_memcpy\_chk` (FORTIFY); the program aborts inside `\\_chk\_fail\_overflow` (exit 133, 5/5 runs) with backtrace `\\_memcpy\_chk <- lab\_parse:46` — the aimed-for write never happens on this build.

5. Rebuild with `clang -g -O0 -D\FORTIFY\_SOURCE=0 ...` and redo the watchpoint: now it fires on the genuine first OOB write, and the three facts a reporter must state can be read off the stopped frame (length, source, destination registers at the `memcpy` call).
6. Lesson recorded live: FORTIFY is a mitigation arm from the lecture's map — it *moves* the crash from your watchpoint address into `libc`'s check. Which build reveals the bug, and which merely reports it after the fact?

Exercise 2 — Confirm the UAF Without ASan

1. Apply the source-auditing lab Exercise A change to the driver (`CREATE "A", CREATE "B", DELETE "A", DELETE "A"`) and rebuild `heap\_plain`. Verified: it still exits 0 without a sanitizer — this is why the debugger, not the exit status, is the instrument.
2. In `lldb`, print the slot pointer (`p g\_strs[0]`) before and after the first `free()`: verified on the reference build, it reads "A" before and "\\x90\\x11" after — allocator freelist bookkeeping overwrote the string in place while the pointer stayed live in `g\_strs`.
3. Set a watchpoint on the freed block's first bytes and confirm: the second `DELETE`'s `find()` reads the corrupted (freed) memory before the second `free()` even happens. This converts the source-auditing lab's ASan observation into instruction-level proof.

Exercise 3 — Breakpoint Script as an API Tracer

1. Script a breakpoint on `str\_store` that prints the command enum and the string argument on every hit, then continues:

```
1 (lldb) b str_store
2 (lldb) breakpoint command add 1
3 > frame variable c
4 > p str
5 > continue
6 > DONE
```
2. Run the four-command driver: the trace is the program's API history without source-level `printf` patching.
3. Do the same for `lab\_parse`: print `size` and the checksum argument of each hit. Count hits over a fuzz run: what is the gate-passing rate? (Compare with the fuzzing lab Exercise 1 numbers.)

Exercise 4 — The Debugger Answers the Fuzzer

1. Take your minimized crasher from the fuzzing lab Exercise 3 and confirm it in a no-sanitizer build under the debugger. Verified on the reference build: the SDK-default build aborts (`SIGABRT`, exit 133, inside `\\_\\_memcpy\\_chk`); the `-D\FORTIFY\_SOURCE=0`

build dies by SIGABRT (exit 134, 3/3 runs) via stack-smash detection — neither segfaults raw. Explain which mitigation arm caught each, and why earlier labs saw different exits.

2. Confirm the corruption itself at the byte level (verified transcript): break at `lab\_target.c:46`, thread until 51; memory read `(uintptr\_t)\&buf[96]` shows 00 80 00 00 before the `memcpy`, 41 41 41 41 after — the payload landed 32 bytes past the buffer end. This is the instruction-level confirmation section for your crash report.

Deliverables

1. Exercise 1's three-fact answer (length/source/destination registers).
2. Exercise 2's watchpoint transcript showing the UAF read.
3. Exercise 3's tracer scripts and hit counts.
4. Exercise 4's confirmation section, appended to the fuzzing-lab report.